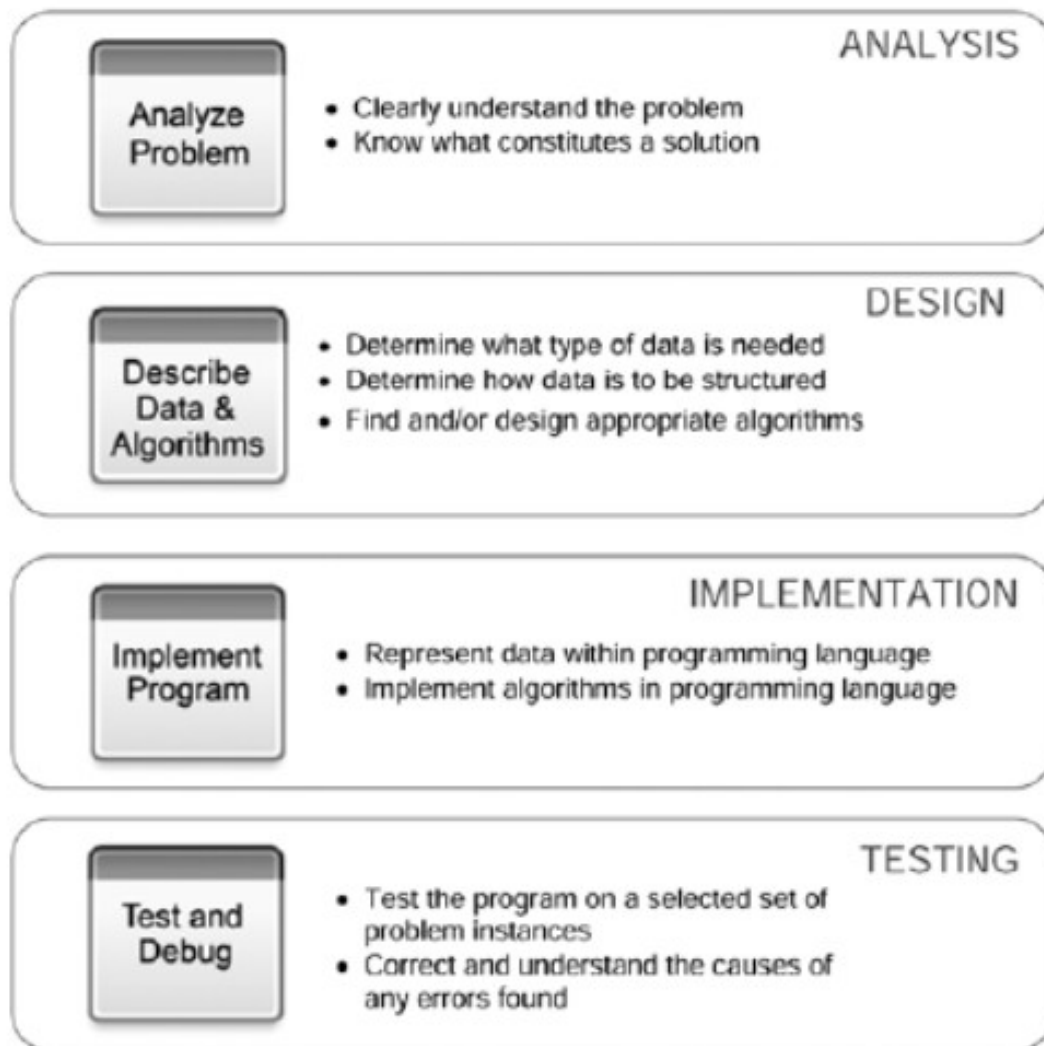
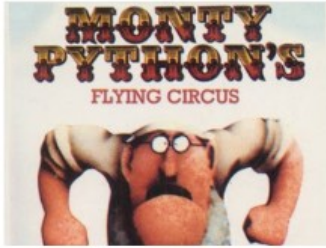


The way of the program: The Python programming language, What is a program?, What is debugging?, Syntax errors, Runtime errors, Semantic errors, Experimental debugging. **Variables, Expressions and Statements:** Values and data types, Variables, Variable names and keywords, Conditional Statements, Evaluating expressions, Operators and operands, Type converter functions, Order of operations, Operations on strings, Input, Composition, The modulus operator. **Iteration:** Assignment, Updating variables, The for loop, The while statement, The Collatz $3n + 1$ sequence, Tables, Nested Loops for Nested Data. **Strings:** Working with strings as single things, Working with the parts of a string, Length, Traversal and the for loop, Slices, String comparison, Strings are immutable, The in and not in operators, A find function, Looping and counting, Optional parameters, The built-in find method, The split method, Cleaning up your strings, The string format method.

Computational Problem solving



Brief History of Python



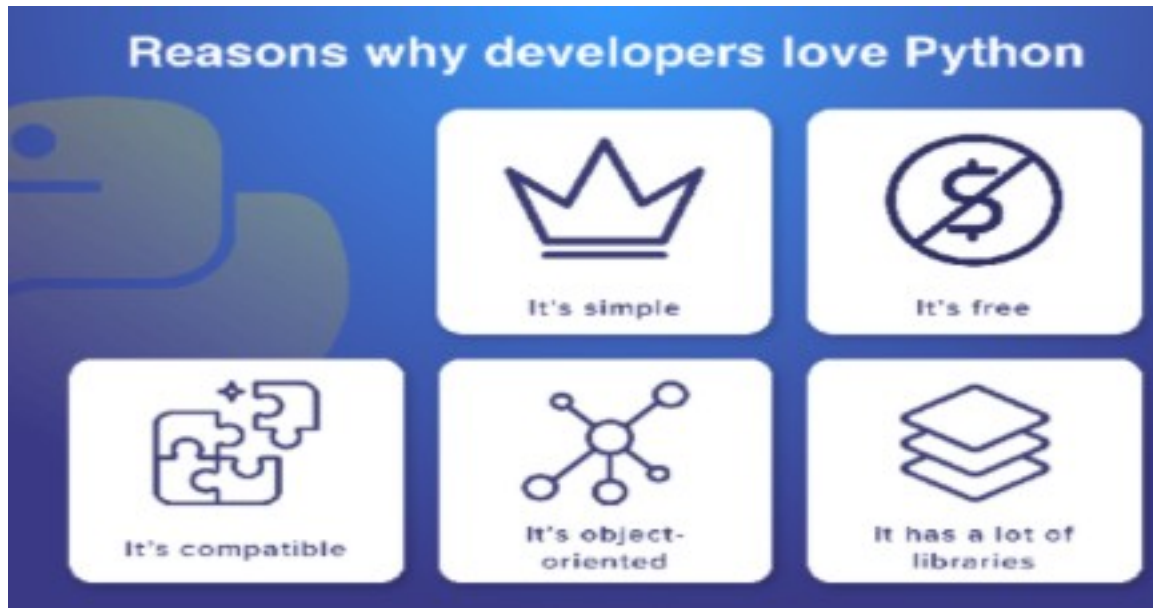
- Guido van Rossum
- Named after Monty Python
- Open sourced from the beginning, managed by Python Software Foundation
- Considered a scripting language, but is much more
- Scalable, object oriented and functional from the beginning
- Used by Google from the beginning



Features of Python



Technical strengths of python



What can i do with Python



Who uses python today?



Comparision with other programming languages

C Program	Java Program	Python Program
<pre>main() { printf("hello, world\n"); }</pre>	<pre>class myfirstjavaprogram { public static void main(String args[]) { System.out.println("Hello World!"); } }</pre>	<pre>print ("Hello World!!")</pre>

	Python	Ruby	JS/ Node.js	C/C++	Java	Go
semicolons	N	N	Y	Y	Y	N
curly braces	N	N*	Y	Y	Y	Y
static types	N	N	N	Y	Y	Y
easy-to-use concurrency	N	N	Y	N	N	Y
multi-core concurrency	N	N	N	Y	Y	Y
compiled	N	N	N	Y	Y	Y
OO: classes, inheritance	Y	Y	Y	Y	Y	N*

The way of the program: The Python programming language, What is a program?, What is debugging?, Syntax errors, Runtime errors, Semantic errors, Experimental debugging.

Variables, Expressions and Statements: Values and data types, Variables, Variable names and keywords, Conditional Statements, Evaluating expressions, Operators and operands, Type converter functions, Order of operations, Operations on strings, Input, Composition, The modulus operator.

Iteration: Assignment, Updating variables, The for loop, The while statement, The Collatz $3n + 1$ sequence, Tables, Two-dimensional tables, Paired Data, Nested Loops for Nested Data.

Strings: Working with strings as single things, Working with the parts of a string, Length, Traversal and the for loop, Slices, String comparison, Strings are immutable, The in and not in operators, A find function, Looping and counting, Optional parameters, The built-in find method, The split method, Cleaning up your strings, The string format method.

The way of the program:

- The Python programming language
- What is a program?
- What is debugging?
- Syntax errors
- Runtime errors
- Semantic errors
- Experimental debugging

The Python programming language

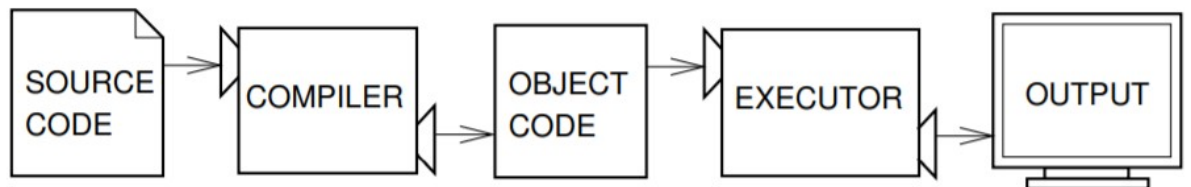
- The programming language you will be learning is **Python**.
- Python is an example of a **high-level language** like C, C++, Perl and Java.
- There are also lowlevel languages, sometimes referred to as “**machine languages**” and “**assembly language**.”
- computers can only **execute** programs written in low-level languages
- programs written in a high-level language have to be **processed** before they can run
- **Disadvantage:** This extra **processing takes some time**, which is a small disadvantage of high-level languages.
- **Advantage:** First, easier to program in a high-level language. Programs written in a high-level language take less time to write, they are shorter and easier to read, and they are more likely to be correct. Second, high-level languages are portable, meaning that they can run on different kinds of computers with few or no modifications.

Two Kinds of Programs process high level languages into low level languages

- **Interpreter:** An interpreter reads a high-level program and executes it, meaning that it does what the program says. It processes the program a little at a time, alternately reading lines and performing computations.



- **Compiler:** A compiler reads the program and translates it completely before the program starts running. In this case, the high-level program is called the source code, and the translated program is called the object code or the executable. Once a program is compiled, you can execute it repeatedly without further translation.



- Python is considered an **interpreted language** because Python programs are **executed by an interpreter**

- There are two ways to use the interpreter: **commandline mode** and **script mode**
- In command-line mode, you type Python programs and the interpreter prints the result
- In script mode, you put all the code in a file and execute

```
print("Hello world")
print(1+1)
print("Hello" * 3)
print(5/3)
print(5//3)

Hello world
2
HelloHelloHello
1.6666666666666667
1

print("Hi Welcome to Python Programming course")

Hi Welcome to Python Programming course

print(5*3)
print("""I am a string""")
print(27//7)
greet="Hello Bob"
x=8
print(greet[x-2])
~3

15
I am a string
3
B

-4

print(5 ** 3)

125
```

What is a program?

- A program is a **sequence of instructions** that specifies how to perform a computation.
- The computation might be **something mathematical**, such as solving a system of equations or finding the roots of a polynomial, but it can also be a symbolic computation, such as searching and replacing text in a document or (strangely enough) compiling a program.
- **input:** Get data from the keyboard, a file, or some other device.
- **output:** Display data on the screen or send data to a file or other device.
- **math:** Perform basic mathematical operations like addition and multiplication.

- **conditional execution:** Check for certain conditions and execute the appropriate sequence of statements.
- **repetition:** Perform some action repeatedly, usually with some variation.

What is debugging?

- Programming is a complex process, and because it is done by human beings, it **often leads to errors**. For whimsical reasons, programming errors are called bugs and the **process of tracking them down and correcting them** is called debugging.
- Three kinds of errors can occur in a program: **syntax errors**, **runtime errors**, and **semantic errors**

Syntax Errors:

- Python can only execute a program if the **program is syntactically correct**; otherwise, the process fails and returns an error message.
- Syntax refers to the **structure of a program** and the rules about that structure

```
try:
    print(5/2)
except:
    print("Exception block")
finally:
    print("Hello finally")

print("Hello")

2.5
Hello finally
Hello
```

Runtime errors:

- the error does not appear **bold text** until you run the program.
- These errors are also called exceptions because they usually indicate that something exceptional (and bad) has happened.

Semantic errors:

- If there is a semantic error in your program, it will run successfully, in the sense that the computer will not generate any error messages, but it will **not do the right thing**.
- It will do something else. Specifically, it will do what you told it to do.

Experimental debugging:

- Once you have an **idea what is going wrong, you modify** your program and try again.
- If your **hypothesis was correct, then you can predict the result** of the modification **bold text**, and you take a step closer to a working program.
- If your **hypothesis was wrong, you have to come up with a new one**.

Variables, Expressions and Statements

- Values and data types
- Variables
- Variable names and keywords
- Conditional Statements
- Evaluating expressions
- Operators and operands
- Type converter functions
- Order of operations
- Operations on strings
- Input
- Composition
- modulus operator

Values and types

```
#This prints a value
print(4)
print(type(4))
#This prints a type of the value
print(type("Hello world"))
print(type(4.5))
print(type(False))

4
<class 'int'>
<class 'str'>
<class 'float'>
<class 'bool'>
```

Variables

- Unlike other programming languages, Python has no command for declaring a variable.
- A variable is created the moment you first assign a value to it.
- Variables do not need to be declared with any particular type and can even change type after they have been set.

```
x = 5
print(type(x))
y = "John"
print(type(y))
print(x)
print(y)
x = 4 # x is of type int
print(type(x))
x = "Sally" # x is now of type str
print(type(x))
```

```

<class 'int'>
<class 'str'>
5
John
<class 'int'>
<class 'str'>

x=5
print(type(x))

<class 'int'>

x="Heaven"
print(type(x))
x=7.5
print(type(x))

<class 'str'>
<class 'float'>

```

Variable names and Keywords

- A variable can have a short name (like x and y) or a more descriptive name (age, carname, total_volume). R
- A variable name must start with a letter or the underscore character
- A variable name cannot start with a number
- variable name can only contain alpha-numeric characters and underscores (A-z, 0-9, and _)
- Variable names are case-sensitive (age, Age and AGE are three different variables)

Keywords: Python has 29 keywords

and	def	exec	if	not	return
assert	del	finally	import	or	try
break	elif	for	in	pass	while
class	else	from	is	print	yield
continue	except	global	lambda	raise	

Statements:

- A statement is an instruction that the Python interpreter can execute. We have seen two kinds of statements: **print and assignment**

```

print(1)
x=2
print(x)

1
2

```

Evaluating Expressions:

- An expression is a combination of **values, variables, and operators**. If you type an expression on the command line, the interpreter evaluates it and displays the result

```
print(1+1)
print(17)
x=5
y=3
print(x-y)
msg="Hello"
concat="world"
print(msg+concat)

2
17
2
Helloworld
```

Operators and Operands:

- Operators are special symbols that represent computations like addition and multiplication. The values the operator uses are called operands

20+32 hour-1 hour*60+minute minute/60 5**2 (5+9)*(15-7)

Order of Operations:

Python follows PEDMAS rule.

- P Parentheses, then
- E Exponents, then
- MD Multiplication and division, left to right, then
- AS Addition and subtraction, left to right

```
#Evaluate the following expression
print(2*(12+1) - (20/4))
#An example of Python exponent
exp1 = 4**3
exp2 = 2**4
exp3 = 5**5
print(exp1)
print("The exponent of 4**3 = ",exp1)
print("The exponent of 2**4 = ",exp2)
print("The exponent of 5**5 = ",exp3)
print("The exponent of exp1,exp2,exp3 = ",exp1,exp2,exp3)
print(pow(3,2))

21.0
64
The exponent of 4**3 = 64
```

```
The exponent of 2**4 = 16
The exponent of 5**5 = 3125
The exponent of exp1,exp2,exp3 = 64 16 3125
9
```

Operations on Strings:

```
fruit = "banana"
bakedGood = " nut bread"
print("After concatenating strings:",fruit+ bakedGood)
```

After concatenating strings: banana nut bread

```
Simple_string = 'Hello String'
Sample_string = "Hello String with double quotes"
print(Simple_string)
print(Sample_string)
```

Hello String
Hello String with double quotes

```
print(2 + 2)
print("Hello" + "World")
print("Hello" + 3)
```

4
HelloWorld

```
-----
-----
TypeError                                Traceback (most recent call
last)
```

```
<ipython-input-27-76550295bbdb> in <module>
      1 print(2 + 2)
      2 print("Hello" + "World")
----> 3 print("Hello" + 3)
```

TypeError: can only concatenate str (not "int") to str

```
print("I 'm happy to share the code")
```

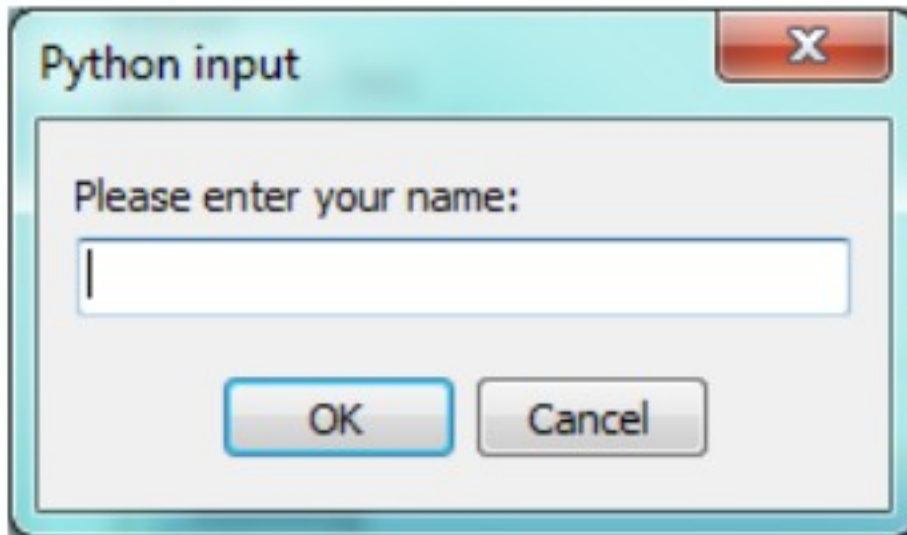
I 'm happy to share the code

```
print('I\'m happy to share')
```

I'm happy to share

Input:

There is a built-in function in Python for getting input from the user: `n = input("Please enter your name: ")`



```
x=input("Enter a number")
print(x)
print(type(x))
y=input()
print(y)
print(type(y))
```

```
Enter a number10
10
<class 'str'>
hi
hi
<class 'str'>
```

```
x = input("Enter x:")
print(x)
print(type(x))
x = float(x)
print(x)
print(type(x))
```

```
Enter x:10
10
<class 'str'>
10.0
<class 'float'>
```

```
x = float(input("Enter x:"))
print(type(x))
```

```
Enter x:10
<class 'float'>
```

```
m,n,o = input("Enter m, n an o:").split(',')
m = int(m)
print(m)
print(n)
print(o)
```

```
Enter m, n an o: 1,2,3
```

```
1
2
3
```

```
help(str)
```

Help on class str in module builtins:

```
class str(object)
|   str(object='') -> str
|   str(bytes_or_buffer[, encoding[, errors]]) -> str
|
|   Create a new string object from the given object. If encoding or
|   errors is specified, then the object must expose a data buffer
|   that will be decoded using the given encoding and error handler.
|   Otherwise, returns the result of object.__str__() (if defined)
|   or repr(object).
|   encoding defaults to sys.getdefaultencoding().
|   errors defaults to 'strict'.
|
|   Methods defined here:
|
|   __add__(self, value, /)
|       Return self+value.
|
|   __contains__(self, key, /)
|       Return key in self.
|
|   __eq__(self, value, /)
|       Return self==value.
|
|   __format__(self, format_spec, /)
|       Return a formatted version of the string as described by
format_spec.
|
|   __ge__(self, value, /)
|       Return self>=value.
```



```
__getattr__(self, name, /)
    Return getattr(self, name).

__getitem__(self, key, /)
    Return self[key].

__getnewargs__(...)

__gt__(self, value, /)
    Return self>value.

__hash__(self, /)
    Return hash(self).

__iter__(self, /)
    Implement iter(self).

__le__(self, value, /)
    Return self<=value.

__len__(self, /)
    Return len(self).

__lt__(self, value, /)
    Return self<value.

__mod__(self, value, /)
    Return self%value.

__mul__(self, value, /)
    Return self*value.

__ne__(self, value, /)
    Return self!=value.

__repr__(self, /)
    Return repr(self).

__rmod__(self, value, /)
    Return value%self.

__rmul__(self, value, /)
    Return value*self.

__sizeof__(self, /)
    Return the size of the string in memory, in bytes.

__str__(self, /)
```

```

    Return str(self).

    capitalize(self, /)
        Return a capitalized version of the string.

        More specifically, make the first character have upper case
and the rest lower
        case.

    casefold(self, /)
        Return a version of the string suitable for caseless
comparisons.

    center(self, width, fillchar=' ', /)
        Return a centered string of length width.

        Padding is done using the specified fill character (default is
a space).

    count(...)
        S.count(sub[, start[, end]]) -> int

        Return the number of non-overlapping occurrences of substring
sub in
        string S[start:end]. Optional arguments start and end are
        interpreted as in slice notation.

    encode(self, /, encoding='utf-8', errors='strict')
        Encode the string using the codec registered for encoding.

        encoding
            The encoding in which to encode the string.
        errors
            The error handling scheme to use for encoding errors.
            The default is 'strict' meaning that encoding errors raise a
            UnicodeEncodeError. Other possible values are 'ignore',
'replace' and
            'xmlcharrefreplace' as well as any other name registered
with
            codecs.register_error that can handle UnicodeEncodeErrors.

    endswith(...)
        S.endswith(suffix[, start[, end]]) -> bool

        Return True if S ends with the specified suffix, False
otherwise.
        With optional start, test S beginning at that position.
        With optional end, stop comparing S at that position.
        suffix can also be a tuple of strings to try.

```

```

    expandtabs(self, /, tabsize=8)
        Return a copy where all tab characters are expanded using
spaces.

        If tabsize is not given, a tab size of 8 characters is
assumed.

    find(...)
        S.find(sub[, start[, end]]) -> int

        Return the lowest index in S where substring sub is found,
such that sub is contained within S[start:end]. Optional
arguments start and end are interpreted as in slice notation.

        Return -1 on failure.

    format(...)
        S.format(*args, **kwargs) -> str

        Return a formatted version of S, using substitutions from args
and kwargs.
        The substitutions are identified by braces ('{' and '}').

    format_map(...)
        S.format_map(mapping) -> str

        Return a formatted version of S, using substitutions from
mapping.
        The substitutions are identified by braces ('{' and '}').

    index(...)
        S.index(sub[, start[, end]]) -> int

        Return the lowest index in S where substring sub is found,
such that sub is contained within S[start:end]. Optional
arguments start and end are interpreted as in slice notation.

        Raises ValueError when the substring is not found.

    isalnum(self, /)
        Return True if the string is an alpha-numeric string, False
otherwise.

        A string is alpha-numeric if all characters in the string are
alpha-numeric and
        there is at least one character in the string.

    isalpha(self, /)

```

Return True if the string is an alphabetic string, False otherwise.

A string is alphabetic if all characters in the string are alphabetic and there is at least one character in the string.

`isascii(self, /)`

Return True if all characters in the string are ASCII, False otherwise.

ASCII characters have code points in the range U+0000-U+007F. Empty string is ASCII too.

`isdecimal(self, /)`

Return True if the string is a decimal string, False otherwise.

A string is a decimal string if all characters in the string are decimal and there is at least one character in the string.

`isdigit(self, /)`

Return True if the string is a digit string, False otherwise.

A string is a digit string if all characters in the string are digits and there is at least one character in the string.

`isidentifier(self, /)`

Return True if the string is a valid Python identifier, False otherwise.

Use `keyword.iskeyword()` to test for reserved identifiers such as "def" and "class".

`islower(self, /)`

Return True if the string is a lowercase string, False otherwise.

A string is lowercase if all cased characters in the string are lowercase and there is at least one cased character in the string.

`isnumeric(self, /)`

Return True if the string is a numeric string, False otherwise.

| A string is numeric if all characters in the string are
numeric and there is at
| least one character in the string.

| `isprintable(self, /)`
| Return True if the string is printable, False otherwise.

| A string is printable if all of its characters are considered
printable in
| `repr()` or if it is empty.

| `isspace(self, /)`
| Return True if the string is a whitespace string, False
otherwise.

| A string is whitespace if all characters in the string are
whitespace and there
| is at least one character in the string.

| `istitle(self, /)`
| Return True if the string is a title-cased string, False
otherwise.

| In a title-cased string, upper- and title-case characters may
only
| follow uncased characters and lowercase characters only cased
ones.

| `isupper(self, /)`
| Return True if the string is an uppercase string, False
otherwise.

| A string is uppercase if all cased characters in the string
are uppercase and
| there is at least one cased character in the string.

| `join(self, iterable, /)`
| Concatenate any number of strings.

| The string whose method is called is inserted in between each
given string.
| The result is returned as a new string.

| Example: `'.'.join(['ab', 'pq', 'rs']) -> 'ab.pq.rs'`

| `ljust(self, width, fillchar=' ', /)`
| Return a left-justified string of length width.

| Padding is done using the specified fill character (default is

a space).

`lower(self, /)`

Return a copy of the string converted to lowercase.

`rstrip(self, chars=None, /)`

Return a copy of the string with leading whitespace removed.

If `chars` is given and not `None`, remove characters in `chars` instead.

`partition(self, sep, /)`

Partition the string into three parts using the given separator.

This will search for the separator in the string. If the separator is found, returns a 3-tuple containing the part before the separator, the separator itself, and the part after it.

If the separator is not found, returns a 3-tuple containing the original string and two empty strings.

`replace(self, old, new, count=-1, /)`

Return a copy with all occurrences of substring `old` replaced by `new`.

`count`

Maximum number of occurrences to replace.

-1 (the default value) means replace all occurrences.

If the optional argument `count` is given, only the first `count` occurrences are replaced.

`rfind(...)`

`S.rfind(sub[, start[, end]]) -> int`

Return the highest index in `S` where substring `sub` is found, such that `sub` is contained within `S[start:end]`. Optional arguments `start` and `end` are interpreted as in slice notation.

Return -1 on failure.

`rindex(...)`

`S.rindex(sub[, start[, end]]) -> int`

Return the highest index in S where substring sub is found, such that sub is contained within S[start:end]. Optional arguments start and end are interpreted as in slice notation.

Raises ValueError when the substring is not found.

`rjust(self, width, fillchar=' ', /)`
Return a right-justified string of length width.

Padding is done using the specified fill character (default is a space).

`rpartition(self, sep, /)`
Partition the string into three parts using the given separator.

This will search for the separator in the string, starting at the end. If the separator is found, returns a 3-tuple containing the part before the separator, the separator itself, and the part after it.

If the separator is not found, returns a 3-tuple containing two empty strings and the original string.

`rsplit(self, /, sep=None, maxsplit=-1)`
Return a list of the words in the string, using sep as the delimiter string.

`sep`
The delimiter according which to split the string.
None (the default value) means split according to any whitespace, and discard empty strings from the result.

`maxsplit`
Maximum number of splits to do.
-1 (the default value) means no limit.

Splits are done starting at the end of the string and working to the front.

`rstrip(self, chars=None, /)`
Return a copy of the string with trailing whitespace removed.

If chars is given and not None, remove characters in chars instead.

`split(self, /, sep=None, maxsplit=-1)`

Return a list of the words in the string, using sep as the delimiter string.

sep
The delimiter according which to split the string.
None (the default value) means split according to any whitespace,
and discard empty strings from the result.
maxsplit
Maximum number of splits to do.
-1 (the default value) means no limit.

splitlines(self, /, keepends=False)
Return a list of the lines in the string, breaking at line boundaries.

Line breaks are not included in the resulting list unless keepends is given and true.

startswith(...)
S.startswith(prefix[, start[, end]]) -> bool

Return True if S starts with the specified prefix, False otherwise.
With optional start, test S beginning at that position.
With optional end, stop comparing S at that position.
prefix can also be a tuple of strings to try.

strip(self, chars=None, /)
Return a copy of the string with leading and trailing whitespace removed.

If chars is given and not None, remove characters in chars instead.

swapcase(self, /)
Convert uppercase characters to lowercase and lowercase characters to uppercase.

title(self, /)
Return a version of the string where each word is titlecased.

More specifically, words start with uppercased characters and all remaining
cased characters have lower case.

translate(self, table, /)
Replace each character in the string using the given

translation table.

 table
 Translation table, which must be a mapping of Unicode
ordinals to
 Unicode ordinals, strings, or None.

 The table must implement lookup/indexing via `__getitem__`, for
instance a
 dictionary or list. If this operation raises `LookupError`, the
character is
 left untouched. Characters mapped to None are deleted.

 upper(self, /)
 Return a copy of the string converted to uppercase.

 zfill(self, width, /)
 Pad a numeric string with zeros on the left, to fill a field
of the given width.

 The string is never truncated.

Static methods defined here:

 __new__(*args, **kwargs) from builtins.type
 Create and return a new object. See `help(type)` for accurate
signature.

 maketrans(x, y=None, z=None, /)
 Return a translation table usable for `str.translate()`.

 If there is only one argument, it must be a dictionary mapping
Unicode
 ordinals (integers) or characters to Unicode ordinals, strings
or None.

 Character keys will be then converted to ordinals.
 If there are two arguments, they must be strings of equal
length, and

 in the resulting dictionary, each character in x will be
mapped to the

 character at the same position in y. If there is a third
argument, it

 must be a string, whose characters will be mapped to None in
the result.

x = 5
y = 10

```
x,y = y,x
print(x,y)

10 5

x,y=12,15
```

Python Numbers and its types

- There are three numeric types in Python: **int**, **float**, **complex**
- Variables of numeric types are created when you assign a value to them

```
x = 1      # int
y = 2.8    # float
z = 1j     # complex
print(x+z)

#print type of a number
print(type(x))
print(type(y))
print(type(z))

(1+1j)
<class 'int'>
<class 'float'>
<class 'complex'>
```

Python Casting

- Specify a **Variable Type**
- There may be times when you want to specify a type on to a variable. This can be done with casting.
- **int()** - constructs an integer number from an integer literal, a float literal (by rounding down to the previous whole number), or a string literal (providing the string represents a whole number)
- **float()** - constructs a float number from an integer literal, a float literal or a string literal (providing the string represents a float or an integer)
- **str()** - constructs a string from a wide variety of data types, including strings, integer literals and float literals

```
x = int(1.5)    # x will be 1
y = float(x)    # y will be 1.0
z = int("3")    # z will be 3
print(type(x))
print(type(y))
print(type(z))
x = float(1)     # x will be 1.0
y = float(2.8)   # y will be 2.8
z = float("3")   # z will be 3.0
print(type(x))
```

```

print(type(y))
print(type(z))
w = float("4.2") # w will be 4.2
x = str("s1") # x will be 's1'
y = str(2) # y will be '2'
z = str(3.0) # z will be '3.0'
print(type(x))
print(type(y))
print(type(z))
print(type(w))

<class 'int'>
<class 'float'>
<class 'int'>
<class 'float'>
<class 'float'>
<class 'float'>
<class 'str'>
<class 'str'>
<class 'str'>
<class 'float'>

x=float(input("Enter a number"))
#x=int(x)
print(type(x))

Enter a number5.5
<class 'float'>

```

Operations on Strings:

H	e	l	l	o		B	o	b
0	1	2	3	4	5	6	7	8

```

greet="Hello Bob"
print(greet)
print(greet[0])
print(greet[2],greet[3],greet[4])
x=8
print(greet[x-2])
print(greet[3:7])

```

```

Hello Bob
H
l l o
B
lo B

Sample = "Python Programming"
print(Sample)

Python Programming
print(Sample[0],Sample[5],Sample[7])

P n P
print(Sample[:]) # Comple String
Python Programming
Sample[0:7]
{"type": "string"}
len(Sample)
18
Sample[3:5]
{"type": "string"}
Sample = "Python Programming"
print(Sample)
Sample[0:15:4]
Python Programming
{"type": "string"}
print(Sample[::-1])
gnimmargorP nohtyP
Sample[-10:-2]
{"type": "string"}
print(Sample)
Python Programming
Sample1 = Sample[0:4]
print(Sample1)
Sample1[3]='g'

```

Pyth

```
-----  
-----  
TypeError                                Traceback (most recent call  
last)
```

```
<ipython-input-83-b60a4fbef95e> in <module>
```

```
1 Sample1 = Sample[0:4]
```

```
2 print(Sample1)
```

```
----> 3 Sample1[3]='g'
```

```
TypeError: 'str' object does not support item assignment
```

```
Text = "Hello"
```

```
print(type(Text))
```

```
#help(Text)
```

```
help(str)
```

```
sample_list = [1,2,3,4,5]
```

```
print(type(sample_list))
```

```
<class 'list'>
```

```
help(list)
```

```
Help on class list in module builtins:
```

```
class list(object)
```

```
    list(iterable=(), /)
```

```
    Built-in mutable sequence.
```

```
    If no argument is given, the constructor creates a new empty list.  
    The argument must be an iterable if specified.
```

```
    Methods defined here:
```

```
    __add__(self, value, /)  
        Return self+value.
```

```
    __contains__(self, key, /)  
        Return key in self.
```

```
    __delitem__(self, key, /)  
        Delete self[key].
```

```
    __eq__(self, value, /)  
        Return self==value.
```

```
    __ge__(self, value, /)  
        Return self>=value.
```

```
__getattr__(self, name, /)
    Return getattr(self, name).

__getitem__(...)
    x.__getitem__(y) <==> x[y]

__gt__(self, value, /)
    Return self>value.

__iadd__(self, value, /)
    Implement self+=value.

__imul__(self, value, /)
    Implement self*=value.

__init__(self, /, *args, **kwargs)
    Initialize self. See help(type(self)) for accurate signature.

__iter__(self, /)
    Implement iter(self).

__le__(self, value, /)
    Return self<=value.

__len__(self, /)
    Return len(self).

__lt__(self, value, /)
    Return self<value.

__mul__(self, value, /)
    Return self*value.

__ne__(self, value, /)
    Return self!=value.

__repr__(self, /)
    Return repr(self).

__reversed__(self, /)
    Return a reverse iterator over the list.

__rmul__(self, value, /)
    Return value*self.

__setitem__(self, key, value, /)
    Set self[key] to value.
```

```
__sizeof__(self, /)
    Return the size of the list in memory, in bytes.

append(self, object, /)
    Append object to the end of the list.

clear(self, /)
    Remove all items from list.

copy(self, /)
    Return a shallow copy of the list.

count(self, value, /)
    Return number of occurrences of value.

extend(self, iterable, /)
    Extend list by appending elements from the iterable.

index(self, value, start=0, stop=9223372036854775807, /)
    Return first index of value.

    Raises ValueError if the value is not present.

insert(self, index, object, /)
    Insert object before index.

pop(self, index=-1, /)
    Remove and return item at index (default last).

    Raises IndexError if list is empty or index is out of range.

remove(self, value, /)
    Remove first occurrence of value.

    Raises ValueError if the value is not present.

reverse(self, /)
    Reverse *IN PLACE*.

sort(self, /, *, key=None, reverse=False)
    Stable sort *IN PLACE*.
```

Static methods defined here:

```
__new__(*args, **kwargs) from builtins.type
    Create and return a new object. See help(type) for accurate
signature.
```

Data and other attributes defined here:

__hash__ = None

Airthmetic and comparision operators

Operator	Description	Example
+, -	Addition, Subtraction	10 -3
*, %	Multiplication, Modulo	27 % 7 Result: 6
/	Division	10/3 3.333333
//	Truncation Division (also known as floor division) The result of the division is truncated to an integer. Works for integers and floating-point numbers as well	27 // 7 Result: 3
+X, -X	Unary minus and Unary plus (Algebraic signs)	-3
~X	Bitwise negation	~3 - 4 Result: -8
**	Exponentiation	10 ** 3 Result: 1000
or, and, not	Boolean Or, Boolean And, Boolean Not	(a or b) and c
in	"Element of"	1 in [3, 2, 1]
<, <=, >, >=, !=, ==	The usual comparison operators	2 <= 3
~, &, ^	Bitwise Or, Bitwise And, Bitwise XOR	6 ^ 3
<<, >>	Shift Operator	6 << 3

```
print(2+3)
print(2-3)
print(2*3)
print(2%3)
print(2/3)
print(2//3)
print(+2)
print(-3)
print(~3-4)
print(3**3)
print(5 or 6 and 0)
print(2<=3)
```



```
print(6^3)
print(6<<3)

5
-1
6
2
0.6666666666666666
0
2
-3
-8
27
5
True
5
48

5 in [1,2,3,4]

False
```

Example Python Programs

1. Printing Hello World

```
#This is a basic hello world program
#Type Hello world in the print statement
print('Hello world')
print("Hello world")

Hello world
Hello world
```

2. Read input data from user

```
#This program demonstrates how to take input from the user.
#We use input() method
name = input("Enter your name")
print(type(name))
print ("Welcome" + " " +name)

Enter your nameRaju
<class 'str'>
Welcome Raju
```

3. Different data types in python

```
#Different data types in Python
```

```

#An integer variable
a=int(input('Enter an integer variable'))
#A float variable
b=float(input('Enter a float variable'))
#A string variable
c=input('Enter the string variable')

print(type(a))
print(type(b))
print(type(c))

# To print the values of a,b,c
print('The value of a is %d' %a)
print('The value of b is %f' %b)
print('The value of c is %s' %c)

#Another way to print is

print('The value of a is '+str(a))
print('The value of b is '+str(b))
print('The value of c is '+c)

#Another way to print
print("The value of a is:",a)
print("The value of b is:",b)
print("The value of c is:",c)

Enter an integer variable10
Enter a float variable1.5
Enter the string variableHi
<class 'int'>
<class 'float'>
<class 'str'>
The value of a is 10
The value of b is 1.500000
The value of c is Hi
The value of a is 10
The value of b is 1.5
The value of c is Hi
The value of a is: 10
The value of b is: 1.5
The value of c is: Hi

```

4. A program to prompt the user for hours and rate per hour to compute gross pay.

- inputs: Enter Hours per week: 35 Enter Rate: 2.75
- Output: Pay: 96.25

```

hrs = input("Enter hours: ")
rt = input("Enter rate: ")
hours = float(hrs)
rate = float(rt)

if hours > 40:
    extra_hours = hours - 40
    extra_rate = rate * 1.5
    pay = rate * hours + (extra_rate * extra_hours)
    print("Pay:", pay)
else:
    pay = rate * hours
    print("Pay:", pay)

Enter hours: 10
Enter rate: 500
Pay: 5000.0

```

5. A program to calculate the compound interest: Read the p, n, r

```

p,n,r,t=input("Enter principal amount, Number of times interest is
compounded in a year, interest rate and number of years ").split()
p=int(p)
n=int(n)
r=int(r)
t=int(t)

#t= int(input('Enter the number of years'))
A= p* (1 + r/n)**(n*t)
print (A)

Enter principal amount, Number of times interest is compounded in a
year, interest rate and number of years 1000 2 10 30
4.887367798068926e+49

x,y=input().split(',')
x=int(x)
y=int(y)
print(type(x),type(y))

1,2
<class 'int'> <class 'int'>

name = 'Mike'
age = 29
print("My name is {} and age is {}".format(name,age))

My name is Mike and age is 29

funds = 150.97823
print("Funds: {}".format(funds))

```

```

print("Funds: {:.f}".format(funds))
print("Funds: {:.1f}".format(funds))
print("Funds: {:10.1f}".format(funds))
print("Funds: {:>10.1f}".format(funds))
print("Funds: {:<10.1f}".format(funds))
print("Funds: {:^-10.1f}".format(funds))

```

```

Funds: 150.97823
Funds: 150.978230
Funds: 151.0
Funds:      151.0
Funds:      151.0
Funds: 151.0
Funds: --151.0--

```

Exercise Programs:

1. Write a Python program that allows the user to enter two integer values, and displays the results when each of the following arithmetic operators are applied. For example, if the user enters the values 7 and 5, the output would be, $7 + 5 = 12$ $7 - 5 = 2$ $7 * 5 = 35$ $7 / 5 = 1.40$ $7 // 5 = 1$ $7 \% 5 = 2$ $7 ** 5 = 16,807$

2. Write a Python program that allows the user to enter any integer value, and displays the value of 2 raised to that power. Your program should function as as given."

input:What power of two? 10 output: Two to the power of 10 is 1024

3. Write a Python program that allows the user to enter any integer base and integer exponent,

and displays the value of the base raised to that exponent. Your program should function as shown below. What base? 10 What power of 10 ? 4 10 to the power of 4 is 10000

4. Write a python program that asks the user how many coins of various types they have, and then prints the total amount of money in rupees

Python Programming Structure

Indentation

- **White space** at the beginning of the line is important, is called indentation.
- **Leading whitespace** (spaces and tabs) at the beginning of the logical line (which understand by python) is used to determine the indentation level of the logical line, which in turn is used to determine the **grouping of statements**.
- **Do not use a mixture** of tabs and spaces for the indentation. It is strongly recommend that **use a single tab or four spaces**.
- The '#' starts a line comment

Iteration:

- Assignment
- Updating variables
- The for loop, The while statement
- The Collatz $3n + 1$ sequence
- Tables, Two-dimensional tables, Paired Data
- Nested Loops for Nested Data.

Assignment and Multiple assignment and updating variables

```
a=5
print(a)
a=7
print(a)
a=7
print(a)
b=a
print(b)
a=3
print(a)
a,b=3,5
print(a,b)
a,b=b,a
print(a,b)

5
7
7
7
```

```
3
3 5
5 3
```

Control flow statements:

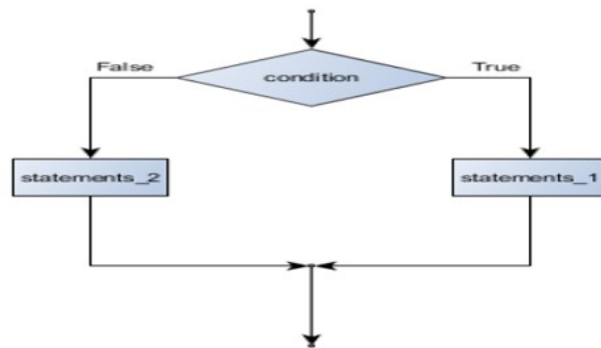
There are 3 control statements: **if**, **for** and **while**.

Simple if statement

The syntax for an **if** statement looks like this:

```
1  if BOOLEAN_EXPRESSION:
2      STATEMENTS_1           # Executed if condition evaluates to True
3  else:
4      STATEMENTS_2           # Executed if condition evaluates to False
```

Flowchart of an if statement with an else clause



Example program to check whether you are eligible to vote or not

```
age=int(input('Enter the age'))
if age >= 18 :
    print('eligible to vote')
    print("End of if statement")
else:
    print("Not eligible to vote")
    print("end of the program")
```

```
Enter the age15
Not eligible to vote
end of the program
```

```
x,y=20,15
```

```
if x > y :
```

```

print("X is greater")
if x > y:
    print("inner if")
else:
    print("Y is greater")

```

X is greater
inner if

Chained conditional statements

<pre> 1 if x < y: 2 STATEMENTS_A 3 elif x > y: 4 STATEMENTS_B 5 else: 6 STATEMENTS_C </pre>	<pre> 1 if choice == "a": 2 function_one() 3 elif choice == "b": 4 function_two() 5 elif choice == "c": 6 function_three() 7 else: 8 print("Invalid choice.") </pre>
---	--

Example program to demonstrate else if ladder

```

n1=int(input('Enter the first number'))
n2=int(input('Enter the second number'))

print('The values entered are')
print(n1,n2)

print('1.Addition 2. Subtraction 3. Multiplication 4. Division')

choice=int(input('Enter the choice'))

if choice==1:
    res=n1+n2
    print('Addition of %d and %d is %d' %(n1,n2,res))

elif choice==2:
    res=n1-n2
    print('Subtraction of %d and %d is %d' %(n1,n2,res))

elif choice==3:
    res=n1*n2

```



```

    print('Multiplication of %d and %d is %d' %(n1,n2,res))
elif choice==4:
    res=n1//n2
    print('Division of %d and %d is %d' %(n1,n2,res))
else:
    print("Invalid")

```

```

Enter the first number10
Enter the second number20
The values entered are
10 20
1.Addition 2. Subtraction 3. Multiplication 4. Division
Enter the choice1
Addition of 10 and 20 is 30

```

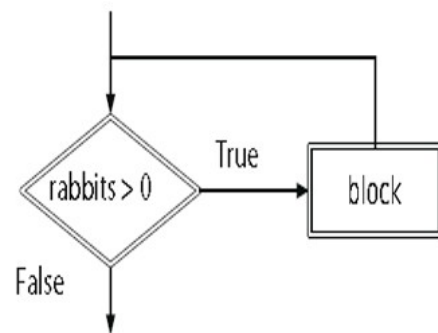
While statements

- **while «*expression*»:**
 – «*block*»

```

>>> rabbits = 3
>>> while rabbits > 0:
...     print(rabbits)
...     rabbits = rabbits - 1
...
3
2
1

```



Example program to print factorial of a number using while loop

```

num = int(input("enter a number: "))

fac = 1
i = 1

while i <= num:
    fac = fac * i
    print(fac)
    i = i + 1

```

```
print("factorial of ", num, " is ", fac)
```

```
enter a number: 5
```

```
1
```

```
2
```

```
6
```

```
24
```

```
120
```

```
factorial of 5 is 120
```

else statements used with while loops

```
#!/usr/bin/python
```

```
count = 0
```

```
while count < 5:
```

```
    print count, " is less than 5"
```

```
    count = count + 1
```

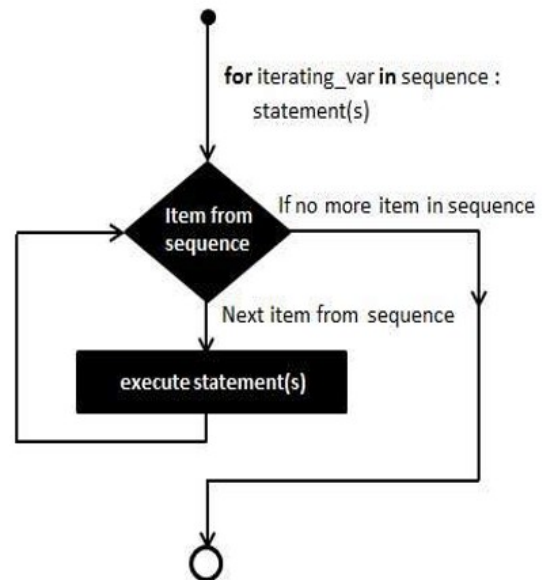
```
else:
```

```
    print count, " is greater than 5"
```

for loop statement

- for “iterating_var” in sequence:
statements(s)

```
>>> for num in range(10):  
...     print(num)  
...  
0  
1  
2  
3  
4  
5  
6  
7  
8  
9
```



```
for letter in 'python':  
    print(letter)  
for i in range(1,5):  
    print(i)  
else:  
    print("For loop is over",i)
```

```
p  
y  
t  
h  
o  
n  
1  
2  
3  
4  
For loop is over 4  
5  
10
```

```
for i in range(5,51,5):  
    print(i)
```

```
15
20
25
30
35
40
45
50
```

Example Python program to display all the prime numbers within an interval

```
# change the values of lower and upper for a different result
lower = 1
upper = 100

# uncomment the following lines to take input from the user
#lower = int(input("Enter lower range: "))
#upper = int(input("Enter upper range: "))

print("Prime numbers between",lower,"and",upper,"are:")

for num in range(lower,upper + 1):
    # prime numbers are greater than 1
    if num > 1:
        for i in range(2,num):
            if (num % i) == 0:
                break
        else:
            print(num)
else:
```

Example python Program to check Armstrong numbers in certain interval

If the given number is equal to the sum of the Nth power of each digit present in that integer, then that number can be an Armstrong Number in Python. For example, 370 is Armstrong Number.

Number of individual digits in 370 = 3

$$370 = 3^3 + 7^3 + 0^3$$

$$= 27 + 343 + 0 = 370$$

```

lower = 100
upper = 900

# To take input from the user
#lower = int(input("Enter lower range: "))
#upper = int(input("Enter upper range: "))

for num in range(lower, upper + 1):

    # order of number
    order = len(str(num))

    # initialize sum
    sum = 0

    # find the sum of the cube of each digit
    temp = num
    while temp > 0:
        digit = temp % 10
        sum += digit ** order
        temp //= 10

    if num == sum:
        print(num)

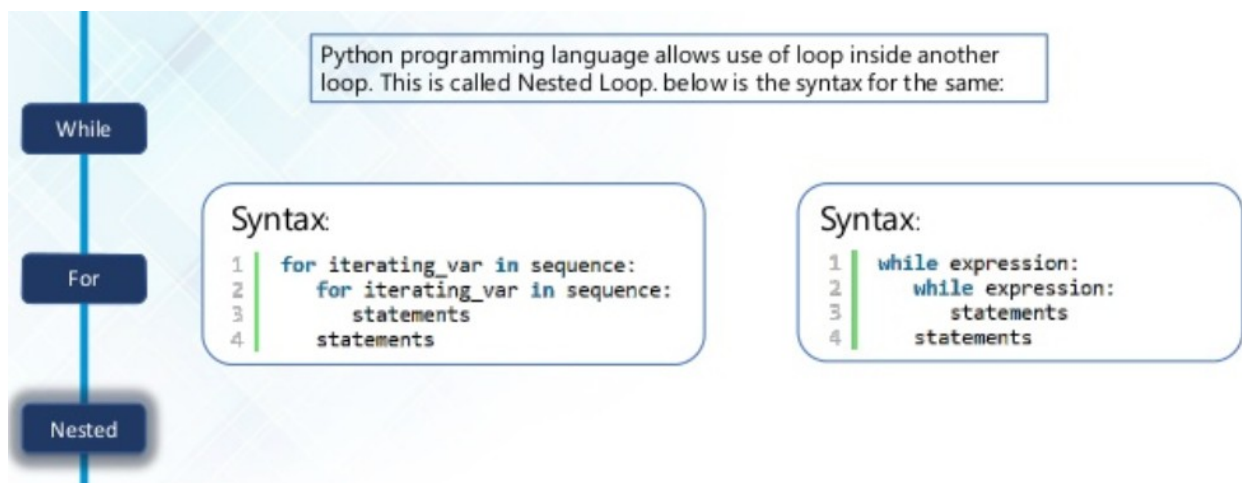
```

```

153
370
371
407

```

Nested loop



```

i=1
while i<=3 :

```

```

print(i,"Outer loop is executed only once")
j=1
while j<=3:
    print(j,"Inner loop is executed until to completion")
    j+=1
i+=1

```

```

1 Outer loop is executed only once
1 Inner loop is executed until to completion
2 Inner loop is executed until to completion
3 Inner loop is executed until to completion
2 Outer loop is executed only once
1 Inner loop is executed until to completion
2 Inner loop is executed until to completion
3 Inner loop is executed until to completion
3 Outer loop is executed only once
1 Inner loop is executed until to completion
2 Inner loop is executed until to completion
3 Inner loop is executed until to completion

```

Example python program to display multiplication table

```

i=1
while i<=5:
    j=1
    while j<=10:
        print(i,"*",j,"=",i*j)
        j+=1
    i+=1
    print("\n")

```

```

1 * 1 = 1
1 * 2 = 2
1 * 3 = 3
1 * 4 = 4
1 * 5 = 5
1 * 6 = 6
1 * 7 = 7
1 * 8 = 8
1 * 9 = 9
1 * 10 = 10

```

```

2 * 1 = 2
2 * 2 = 4
2 * 3 = 6
2 * 4 = 8
2 * 5 = 10
2 * 6 = 12
2 * 7 = 14

```

```
2 * 8 = 16
2 * 9 = 18
2 * 10 = 20
```

```
3 * 1 = 3
3 * 2 = 6
3 * 3 = 9
3 * 4 = 12
3 * 5 = 15
3 * 6 = 18
3 * 7 = 21
3 * 8 = 24
3 * 9 = 27
3 * 10 = 30
```

```
4 * 1 = 4
4 * 2 = 8
4 * 3 = 12
4 * 4 = 16
4 * 5 = 20
4 * 6 = 24
4 * 7 = 28
4 * 8 = 32
4 * 9 = 36
4 * 10 = 40
```

```
5 * 1 = 5
5 * 2 = 10
5 * 3 = 15
5 * 4 = 20
5 * 5 = 25
5 * 6 = 30
5 * 7 = 35
5 * 8 = 40
5 * 9 = 45
5 * 10 = 50
```

Python program to display multiplication table using nested for loop

```
for i in range(1,6): #outer for loop
    print(" ")
    for j in range(1,11): #inner for loop
        print(i*j)
```

1
2
3
4
5
6
7
8
9
10

2
4
6
8
10
12
14
16
18
20

3
6
9
12
15
18
21
24
27
30

4
8
12
16
20
24
28
32
36
40

5
10
15
20
25

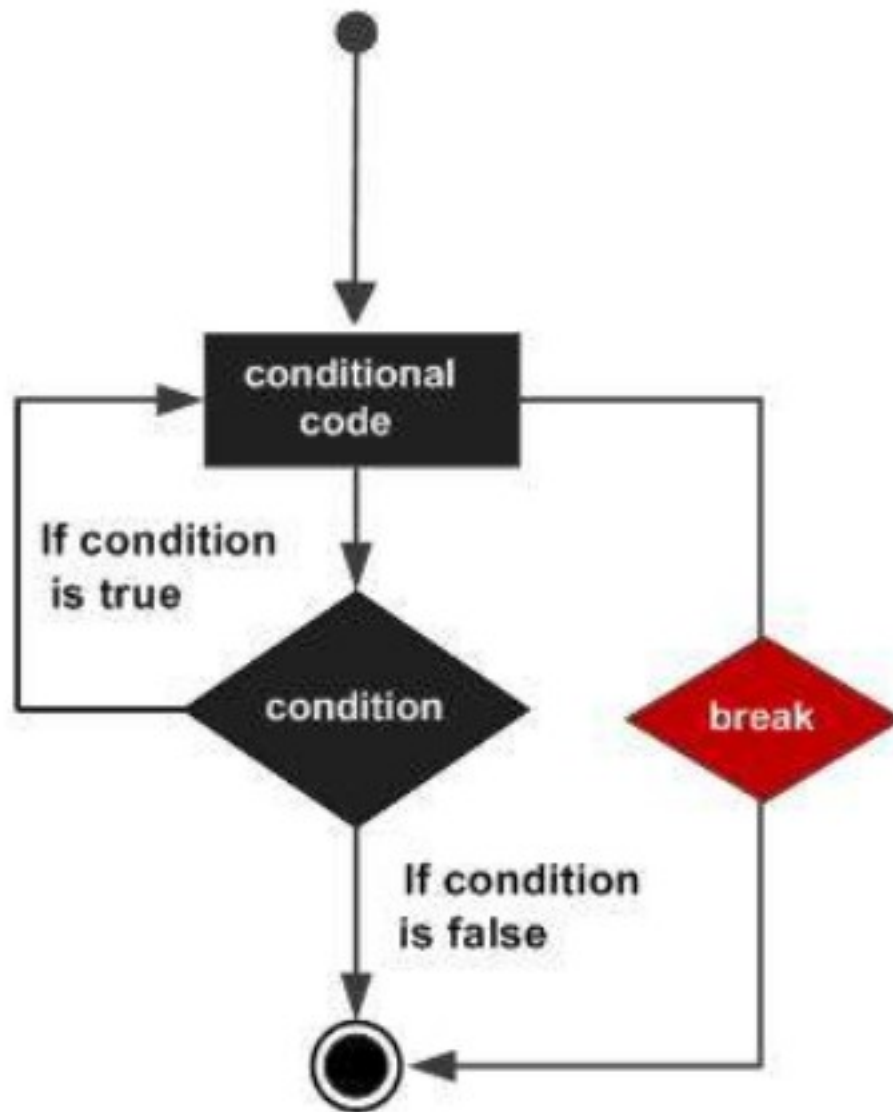

```
30
35
40
45
50
```

Loop Control Statements

- **break statement**: Terminates the **loop** statement and transfers execution to the statement immediately following the loop.
- **continue statement**: Causes the loop to skip the remainder of its body and immediately retest its condition prior to reiterating.
- **pass statement**: The pass statement in Python is used when a statement is required syntactically but you do not want any command or code to execute.

break statement

Flow diagram:



```
for letter in 'Python':    # First Example
    if letter == 'h':
        pass
    print('Current Letter :', letter)
print("End of the program")
```

```
Current Letter : P
Current Letter : y
Current Letter : t
Current Letter : h
Current Letter : o
```

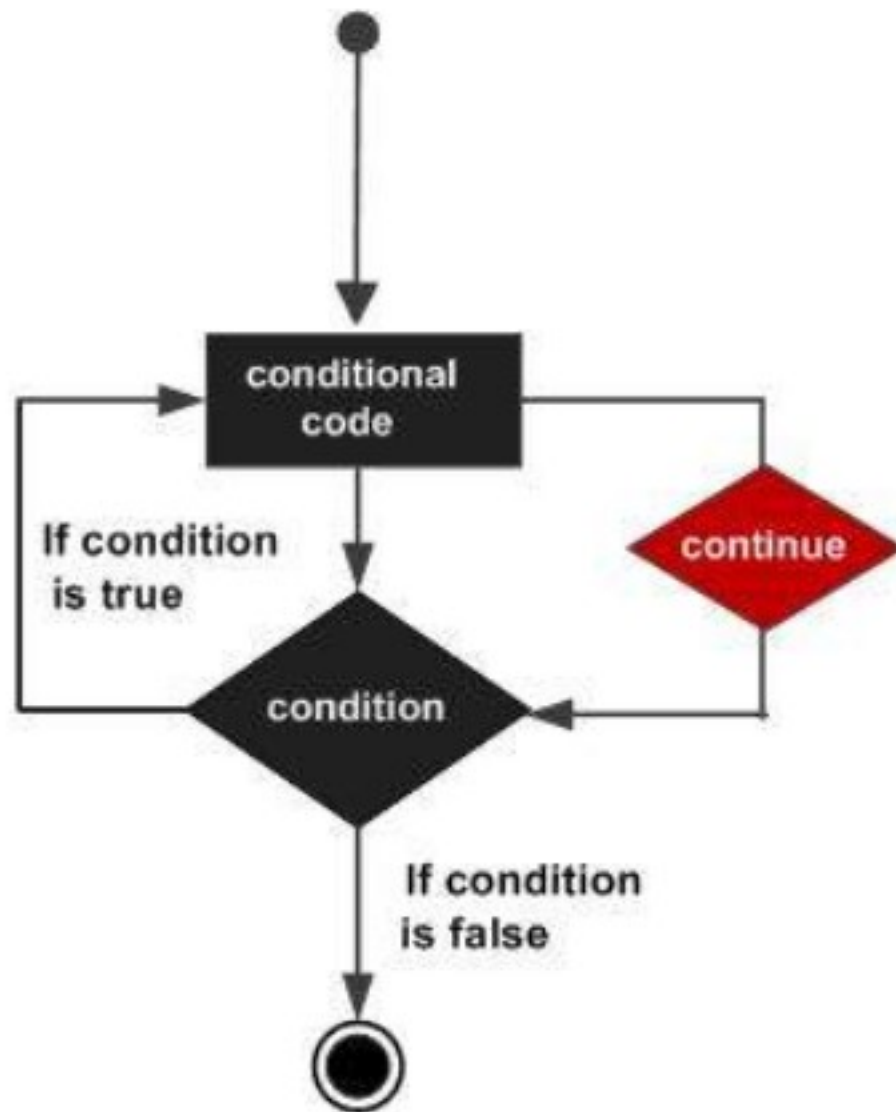
Current Letter : n
End of the program

```
var = 10                                # Second Example
while var > 0:
    print('Current variable value :', var)
    var = var - 1
    if var == 5:
        break

print("Good bye!")
```

continue statement

Flow diagram:



```
for letter in 'Python':    # First Example
    if letter == 'h':
        continue
    print('Current Letter :', letter)

var = 10                    # Second Example
while var > 0:
    var = var - 1
    if var == 5:
        continue
    print('Current variable value :', var)
print("Good bye!")
```

pass statement

pass statement is a null operation, nothing happens when it executes. it is often said to be a placeholder, for code which will be added later.

```
for letter in 'Python':
    if letter == 'h':
        pass
    print('This is pass block')
print('Current Letter :', letter)

print("Good bye!")

Current Letter : P
Current Letter : y
Current Letter : t
This is pass block
Current Letter : h
Current Letter : o
Current Letter : n
Good bye!

x,y = 10,20

for i in "python":
    if x > y:
        print("X is greater")
        break
```

range() in python

- **The range function has many versions.**
 - **range(a)**
 - **range(a, b, k)**
- The first number in the sequence is **a**.
- **b is the limit.**
- **Each successive number in the sequence will increase by the step value k.**

```
number=0
sum=0
```

```

for count in range(5):
    number=int(input("Enter an integer:"))
    sum=sum+number
print("Sum:",sum)
print("count",count)

```

```

count=0
for count in range(1,10):
    sum=sum+count
print("Sum:",sum)
print("count",count)

```

```

for i in range(0, 31, 3):
    print(i,end=" ")
print()

```

```

# using range to print number
# divisible by 5
for i in range(0, 50, 5):
    print(i, end = " ")

```

```

Enter an integer:1
Enter an integer:2
Enter an integer:3
Enter an integer:4
Enter an integer:5
Sum: 15
count 4
Sum: 60
count 9
0 3 6 9 12 15 18 21 24 27 30
0 5 10 15 20 25 30 35 40 45

```

```

for i in range(3, 31, 3):
    print(i,end="-")
print()

```

```

3-6-9-12-15-18-21-24-27-30-

```

```

for i in range(0, 50, 5):
    print(i,end="\t")

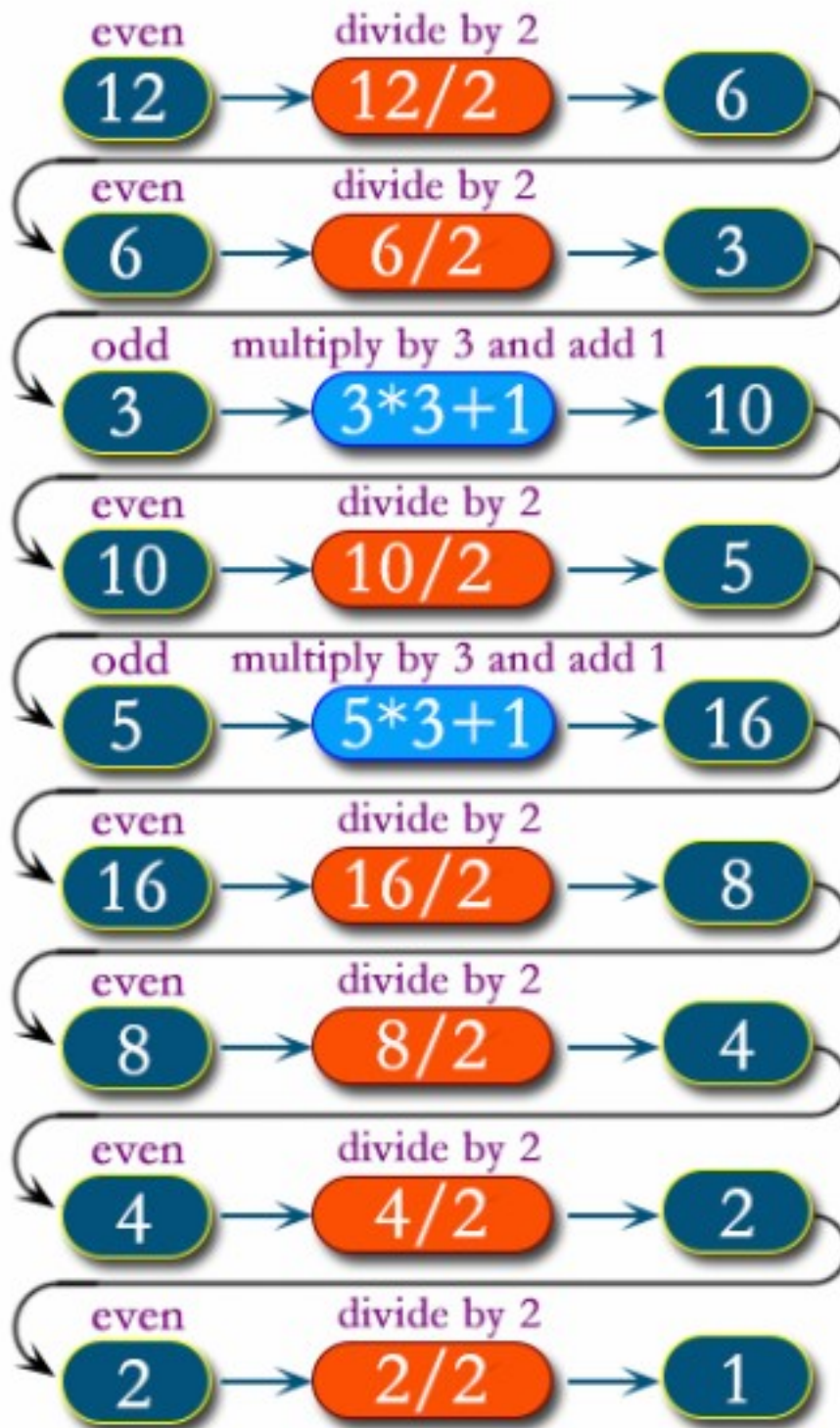
```

```

0      5      10     15     20     25     30     35     40     45

```

The Collatz 3n + 1 sequence



```
n=int(input("Enter number:"))  
print("collatz conjutere sequence of a number is:")
```

```

while n>1:
    if n%2:
        #n is odd
        n=3*n+1
    else:
        #n is even
        n=n//2
    print(n,end=" ")

```

Enter number:15

collatz conjutue sequence of a number is:

46 23 70 35 106 53 160 80 40 20 10 5 16 8 4 2 1

Tables

```

print("n", '\t', "2**n")      #table column headings
print("----", '\t', "-----")

for x in range(13):          # generate values for columns
    print(x, '\t', 2 ** x)

```

n	2**n
---	-----
0	1
1	2
2	4
3	8
4	16
5	32
6	64
7	128
8	256
9	512
10	1024
11	2048
12	4096

Example programs

1. Print First 10 natural numbers using while loop

```

i = 0
while i <= 10:
    print(i)
    i += 1

```

0
1
2


```
3
4
5
6
7
8
9
10
```

```
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
```

2. Print the following pattern

```
print("Second Number Pattern ")
lastNumber = 6
for row in range(1, lastNumber):
    for column in range(1, row + 1):
        print(column, end=' ')
    print("")
```

Second Number Pattern

```
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
```

3. Accept number from user and calculate the sum of all number between 1 and given number
For example user given 10 so the output should be 55

```
sum1 = 0
n = int(input("Please enter number "))
for i in range(1, n + 1, 1):
    sum1 += i
print("\n")
print("Sum is: ", sum1)
```

Please enter number 10

Sum is: 55

4. Given a number count the total number of digits in a number For example, the number is 75869, so the output should be 5.

```
num = 75869
count = 0
while num != 0:
    num //= 10
    print(num)
    count += 1
print("Total digits are: ", count)
```

```
7586
758
75
7
0
Total digits are: 5
```

```
5 4 3 2 1
4 3 2 1
3 2 1
2 1
1
```

5. Print the following pattern using for loop

```
n = 5
k = 5
for i in range(0,n+1):
    for j in range(k-i,0,-1):
        print(j,end=' ')
    print()
```

```
5 4 3 2 1
4 3 2 1
3 2 1
2 1
1
```

6. Display -10 to -1 using for loop

```
for num in range(-10, 0):  
    print(num)
```

-10
-9
-8
-7
-6
-5
-4
-3
-2
-1

Exercise Programs

1. Write a script that asks a user for a number. The script adds 3 to that number. Then multiplies the result by 2, subtracts 4, subtracts twice the original number, adds 3, then prints the result.

1. Write a program that asks users for their favorite color. Create the following output (assuming "red" is the chosen color). Use "+" and "*".

[illegible]

1. **Write a python program for the following cases.** **case 1:** which asks someone to input their first name, last name and phone number. If the user does not type at least some characters for each of these, print "Do not leave any fields empty" otherwise print "Thank you". (Hint: if a variable is empty, its value will be "false".) **case 2:** Change the script so that the script prints "Thank you" if either the first name or the last name or the phone number is supplied. Print "Do not leave all fields empty" otherwise. **case 3:** Change the script so that only first name and last name are required. The phone number is optional.

1. Write a program that asks a user to input a color. If the color is black or white, output "The color was black or white". If it starts with a letter that comes after "k" in the alphabet, output "The color starts with a letter that comes after 'k' in the alphabet". (Optional: consider both capitalized and non-capitalized words. Note: the order of the alphabet in Unix and Python is: symbols, numbers, upper case letters, lower case letters.)

1. Write a program that asks a user to input a number. If the number equals "5", output "My lucky number". If the number is larger than 10, output "What a large number!". In all other cases, output "That's not my lucky number."

1. Write a program that asks five times to guess the lucky number. Use a while loop and a counter. for the cases given below: **case 1:** The program asks for five guesses (no matter whether the correct number was guessed or not).If the correct number is guessed, the program outputs "Good guess!",otherwise it outputs "Try again!". After the fifth guess it stops and prints "Game over."

case 2: break: In the previous example, insert "break" after the "Good guess!" print statement. "break" will terminate the while loop so that users do not have to continue guessing after they found the number. If the user does not guess the number at all print "Sorry but that was not very successful" (use "else" for this)

case 3: Counting hits: Modify the program again. This time the program continues even after the correct number was guessed but it counts how often the correct number was guessed. You'll need two counters: one for the while loop and another one for the number of correct guesses. After the while loop is finished, use an if statement to print either "You guessed the number ... times" or "The number was not guessed at all".

1. Print all multiples of 13 that are smaller than 100. Use the range function in the following manner: `range(start, end, step)` where "start" is the starting value of the counter, "end" is the end value and "step" is the amount by which the counter is increased each time

Strings

How to define strings:

- Strings are amongst the most popular types in Python.
- We can create them simply by enclosing characters in quotes.
- **Python treats single quotes the same as double quotes.**
- Creating strings is as simple as assigning a value to a variable.
- For example:

var1 = 'Hello World!'

var2 = "Python Programming"

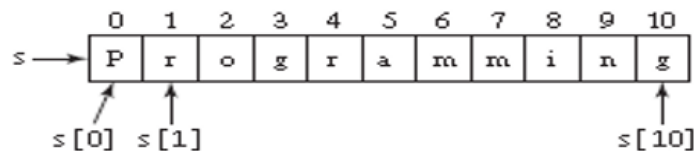
String operators

Operator	Description	Example
+	Concatenation - Adds values on either side of the operator	a + b will give HelloPython
*	Repetition - Creates new strings, concatenating multiple copies of the same string	a*2 will give -HelloHello
[]	Slice - Gives the character from the given index	a[1] will give e
[:]	Range Slice - Gives the characters from the given range	a[1:4] will give ell
in	Membership - Returns true if a character exists in the given string	H in a will give 1
not in	Membership - Returns true if a character does not exist in the given string	M not in a will give 1
r/R	Raw String - Suppress actual meaning of Escape characters. The syntax for raw strings is exactly the same as for normal strings with the exception of the raw string operator, the letter "r," which precedes the quotation marks. The "r" can be lowercase (r) or uppercase (R) and must be placed immediately preceding the first quote mark.	print r'\n' prints \n and print R'\n' prints \n <pre>>>> plain_str='newline \n' >>> raw_str = r'newline \n' >>> print (plain_str) newline >>> print (raw_str) newline \n >>></pre>

Index operator

A string is a sequence of characters. A character in the string can be accessed through the index operator using the syntax:

- `s[index]`
- The indexes are **0 based**; that is, they range from 0 to `len(s)-1`,



Slicing operator

The slicing operator returns a slice of the string using the syntax
`s[start : end]`

```
s="Hello"  
print(s[1:4])  
print(s[-1])  
  
ell  
o
```

The Concatenation and repetition operator:

You can join, or concatenate, two strings by using the concatenation operator (+).

You can also use the repetition operator (*) to concatenate the same string multiple times

```
s1="welcome"  
s2="python"  
s3=s1+" "+ "to"+" "+s2  
print(s3)  
s4=3*s1  
print(s4)
```



```
welcome to python  
welcomewelcomewelcome
```

in and not in operator

You can use the **in** and **not in** operators to test whether a string is in another string.

```
s1="welcome"  
print("come" in s1)  
print("come" not in s1)  
  
True  
False
```

Comparing strings

You can compare strings by using the comparison operators (**=**, **!=**, **>**, **>=**, **<**, and **<=**)

```
print("green"=="glow")  
print("green"!="glow")  
print("green">"glow")  
print("green">="glow")  
print("green"<"glow")  
print("green"<="glow")  
  
False  
True  
True  
True  
False  
False
```

Testing strings

str	
isalnum(): bool	Returns True if characters in this string are alphanumeric and there is at least one character.
isalpha(): bool	Returns True if characters in this string are alphabetic and there is at least one character.
isdigit(): bool	Returns True if this string contains only number characters.
isidentifier(): bool	Returns True if this string is a Python identifier.
islower(): bool	Returns True if all characters in this string are lowercase letters and there is at least one character.
isupper(): bool	Returns True if all characters in this string are uppercase letters and there is at least one character.
isspace(): bool	Returns True if this string contains only whitespace characters.

```
str="this2020"  
print(str.isalnum())  
print(str.isalpha())  
str="123"  
print(str.isdigit())  
str="if"  
print(str.isidentifier())  
print(str.islower())  
print(str.isupper())  
str=" "  
print(str.isspace())
```

```
True  
False  
True  
True  
True  
False  
True
```

Searching for sub strings

str	
<code>endswith(s1: str): bool</code>	Returns True if the string ends with the substring s1.
<code>startswith(s1: str): bool</code>	Returns True if the string starts with the substring s1.
<code>find(s1): int</code>	Returns the lowest index where s1 starts in this string, or -1 if s1 is not found in this string.
<code>rfind(s1): int</code>	Returns the highest index where s1 starts in this string, or -1 if s1 is not found in this string.
<code>count(substring): int</code>	Returns the number of non-overlapping occurrences of this substring.

```
s="Welcome to python"
print(s.endswith("thon"))
print(s.startswith("good"))
print(s.find("become"))
print(s.rfind("o"))
print(s.find("o"))
print(s.count("o"))

True
False
-1
15
4
3

str="this is string example.....wow!!!"
suffix="wow!!!"
print(str.endswith(suffix))
print(str.endswith(suffix,20))

suffix="is"
print(str.endswith(suffix,2,4))
print(str.endswith(suffix,2,6))

True
True
True
False

str1="this is string example.....wow!!!"
str2="exam"
print(str1.find(str2))
print(str1.find(str2,16))
print(str1.find(str2,40))
```

```
15  
-1  
-1
```

```
str1="this is string example.....wow!!! is good is at best"  
print(str1.replace("is","was"))  
print(str1.replace("is","was",3))  
print(str1)
```

```
thwas was string example.....wow!!! was good was at best  
thwas was string example.....wow!!! was good is at best  
this is string example.....wow!!! is good is at best
```

Converting Strings

str	
capitalize(): str	Returns a copy of this string with only the first character capitalized.
lower(): str	Returns a copy of this string with all letters converted to lowercase.
upper(): str	Returns a copy of this string with all letters converted to uppercase.
title(): str	Returns a copy of this string with the first letter capitalized in each word.
swapcase(): str	Returns a copy of this string in which lowercase letters are converted to uppercase and uppercase to lowercase.
replace(old, new): str	Returns a new string that replaces all the occurrences of the old string with a new string.

```
str="hello world"  
print(str.capitalize())  
print(str.lower())  
print(str.upper())  
print(str.title())  
print(str.swapcase())  
print(str.replace("o","w"))
```

```
Hello world  
hello world  
HELLO WORLD  
Hello World  
HELLO WORLD  
helloworld
```

Stripping whitespaces from a string

str	
<code>lstrip(): str</code>	Returns a string with the leading whitespace characters removed.
<code>rstrip(): str</code>	Returns a string with the trailing whitespace characters removed.
<code>strip(): str</code>	Returns a string with the starting and trailing whitespace characters removed.

```
str="    Hello, Welcome to python    "  
print(str.lstrip())  
print(str.rstrip())  
print(str.strip())
```

```
Hello, Welcome to python  
    Hello, Welcome to python  
Hello, Welcome to python
```

String formatting operator: %

- One of Python's coolest features is the string format operator %.
- This operator is unique to strings and makes up for the pack of having functions from C's `printf()` family

```
print("My name is %s and age is %d"%( 'Raju',21))
```

```
My name is Raju and age is 21
```

String and secret codes

- The **ord** function returns the numeric (ordinal) code of a single character.
- The **chr** function converts a numeric code to the corresponding character.

```
print(ord('A'))  
print(ord('a'))  
print(chr(97))  
print(chr(52))
```

```
65  
97  
a  
4
```

split method

```
print("Hello python, welcome to strings".split('o'))  
['Hell', ' pyth', 'n, welc', 'me t', ' strings']  
  
x,y=input("Enter x and y:").split()  
print(x,type(x))  
print(y,type(y))  
x,y=input("Enter x and y:").split(',')  
print(x,type(x))  
print(y,type(y))  
  
Enter x and y:china, nepal  
china, <class 'str'>  
nepal <class 'str'>  
Enter x and y:china,nepal  
china <class 'str'>  
nepal <class 'str'>
```

center method

The method **center()** returns centered in a string of length *width*. Padding is done using the specified *fillchar*. Default filler is a space.

```
str="this is string example.....wow!"  
print("str.center(40,'a')",str.center(40,'-'))  
print(str.center(40))  
  
str.center(40,'a') ----this is string example.....wow!-----  
this is string example.....wow!
```

Example Programs

1. Basic String Operations

```
s1 = 'Hello World!'  
s2 = "Python Programming"  
  
print('The index operator on the string can be used as')  
print('The 3rd character in Hello World! is')  
print(s1[2])  
  
print('The 3rd character of Python Programming is ')  
#Please fill the code here  
print(s2[2])
```

```

print('The last character of Hello World! is')
print(s1[-1])

print('The last character of Python Programming is ')
#Please fill the code here
print(s2[-1])

print('Concatenation of two strings is')
print(s1+s2)

print('Use of * operator')
print(s1*3)

print('Use of in operator')
print("He" in s1)

#use of Sting formatting operator(%) like the access specifiers in C
print("My name is: %s and age is: %d"%('hari',27))

```

```

The index operator on the string can be used as
The 3rd character in Hello World! is
l
The 3rd character of Python Programming is
t
The last character of Hello World! is
!
The last character of Python Programming is
g
Concatenation of two strings is
Hello World!Python Programming
Use of * operator
Hello World!Hello World!Hello World!
Use of in operator
True
My name is: hari and age is: 27

```

2. Testing strings

```

s1='Welcome to python'
print('Checking whether string is alphanumeric')
print(s1.isalnum())

s2='Welcome'
print('Checking whether string contains alphabets')
print(s2.isalpha())

s3='2012'

```

```

print('Checking whether string is a digit')
print(s3.isdigit())

str1 = "this2009"
#Check whether the string is alphanumeric
print('Checking whether the string is alphanumeric or not')

#Check whether string is in lower case
print('Checking whether string is in lower case or not')

#Check whether string is in uppercase
print('Checking whether string is in uppercase or not')

str2 = "this is string example....wow!!!"
#Please do the same for the str2 now.

Checking whether string is alphanumeric
False
Checking whether string contains alphabets
True
Checking whether string is a digit
True
Checking whether the string is alphanumeric or not
Checking whether string is in lower case or not
Checking whether string is in uppercase or not

```

3. Searching sub strings

```

str = "this is string example....wow!!!"

print('Usage of endswith()')
suffix = "wow!!!"
print(str.endswith(suffix))

print('Usage of startswith()')
print(str.startswith('this'))

print('Usage of count()')
print(str.count('s'))

print('Usage of find()')
print(str.find('example'))

newstr='Python Programming is fun'
#Use endswith(),startswith(),count() and find() methods for new string newstr
#Fill the code here

```



```
Usage of endswith()
True
Usage of startswith()
True
Usage of count()
3
Usage of find()
15
```

4. Converting strings

```
s1='welcome to python'

print('Usage of capitalize')
print(s1.capitalize())

print('Usage of title()')
print(s1.title())

s2='New England'

print('Usage of upper')
print(s2.upper())

print('Usage of lower')
print(s2.lower())

print('Usage of swapcase')
print(s2.swapcase())

print('Usage of replace')
s3=s2.replace('England', 'Heaven')
print(s3)

Usage of capitalize
Welcome to python
Usage of title()
Welcome To Python
Usage of upper
NEW ENGLAND
Usage of lower
new england
Usage of swapcase
nEW eNGLAND
Usage of replace
New Heaven
```

5. Stripping whitespaces

```
s = " Welcome to Python\t"

print('After removing leading whitespaces')
s1 = s.lstrip()
print(s1)

print('After removing trailing whitespaces')
s2 = s.rstrip()
print(s2)

print('After removing leading and trailing whitespaces')
s3 = s.strip()
print(s3)

newstr='    Hello how are you?    '
#Use the functions above for the new string

After removing leading whitespaces
Welcome to Python
After removing trailing whitespaces
 Welcome to Python
After removing leading and trailing whitespaces
Welcome to Python
```

6. eval function

```
num1=eval(input('Enter a number'))
print(num1)
res=eval('num1+1')
print(res)

Enter a number10
10
11

print(repr("Hello"))

'Hello'
```

Exercise programs

1.Basic Strings

```
#Consider, s1 and s2 are two strings.
# Include two strings s1 and s2
s1 = "programming 101"
s2 = "programming is fun"

print("The strings are initialized as follows")
#Print both the strings
```

```
#Write the expression for  
#Print a string as "programming 101 and programming is fun"  
print('The concatenation of two strings are')  
  
#Print only "programming" from s1  
#Slice the string s1 for extracting only programming
```

****2.** Write a python program that takes a string and prints the frequency of the characters contained in it. For Example:

- Input: Hello
- Output: H:1 e:1 l:2 o:1

1. The third person singular verb form in English is distinguished by the suffix -s, which is added to the stem of the infinitive form: "run -> runs". A simple set of rules can be given as follows: # If the verb ends in "y", remove it and add "ies" # If the verb ends in "o, ch, s, sh, x or z," add "es" # By default just add "s" Your task in this exercise is to provide given a verb in infinitive form returns its third person singular form. Test your function with words like try, brush, run and fix.**Note:** however that the rules must be regarded as heuristic, in the sense that you must not expect them to work for all cases. **Tip:** Check out the string method "endswith()".

4. Write a Python program to reverse a string. Sample String : "1234abcd" Expected Output : "dcba4321"

[link text](#) 5. Write a python program to check string is palindrome or not

6. Write a program to print every character of a string entered by user in a new line using loop and also construct a string from all those characters entered by user and print the length of the string without using len() function.

7. Write a program to find the first and the last occurrence of the letter 'o' and character ',' in "Hello, World".

8. Write a program that takes your full name as input and displays the abbreviations of the first and middle names except the last name which is displayed as it is. For example, if your name is Robert Brett Roser, then the output should be R.B.Roser.

9. Write a program to find the number of vowels, consonants, digits and white space characters in a string.

10. Write a program to make a new string with all the consonants eliminated from the string read from the user. For example

- Input: Hello, have a good day
- Output: Hll, hv gd dy **bold text**

Glossary

- **Problem solving:** The process of formulating a problem, finding a solution, and expressing the solution.
 - **High-level language:** A programming language like Python that is designed to be easy for humans to read and write.
 - **low-level language:** A programming language that is designed to be easy for a computer to execute; also called "machine language" or "assembly language."
 - **Portability:** A property of a program that can run on more than one kind of computer.
 - **Interpret:** To execute a program in a high-level language by translating it one line at a time.
 - **Compile:** To translate a program written in a high-level language into a low level language all at once, in preparation for later execution.
 - **Source code:** A program in a high-level language before being compiled.
 - **Object code:** The output of the compiler after it translates the program.
 - **Executable:** Another name for object code that is ready to be executed.
 - **Script:** A program stored in a file (usually one that will be interpreted).
 - **Program:** A set of instructions that specifies a computation.
 - **Algorithm:** A general process for solving a category of problems.
 - **Bug:** An error in a program.
 - **Debugging:** The process of finding and removing any of the three kinds of programming errors.
 - **Syntax:** The structure of a program.
 - **Syntax error:** An error in a program that makes it impossible to parse (and therefore impossible to interpret).
 - **Runtime error:** An error that does not occur until the program has started to execute but that prevents the program from continuing.
 - **Exception:** Another name for a runtime error.
-

- **Semantic error:** An error in a program that makes it do something other than what the programmer intended.
 - **Semantics:** The meaning of a program.
 - **Natural language:** Any one of the languages that people speak that evolved naturally.
 - **formal language:** Any one of the languages that people have designed for specific purposes, such as representing mathematical ideas or computer programs; all programming languages are formal languages.
 - **Token:** One of the basic elements of the syntactic structure of a program, analogous to a word in a natural language.
 - **Parse:** To examine a program and analyze the syntactic structure.
 - **Print statement:** An instruction that causes the Python interpreter to display a value on the screen.
 - **Value:** A number or string (or other thing to be named later) that can be stored in a variable or computed in an expression.
-

- **Type:** A set of values. The type of a value determines how it can be used in expressions. So far, the types you have seen are integers (type `int`), floating-point numbers (type `float`), and strings (type `string`).
- **Floating-point:** A format for representing numbers with fractional parts.
- **Variable:** A name that refers to a value.
- **Statement:** A section of code that represents a command or action. So far, the statements you have seen are assignments and print statements.
- **Assignment:** A statement that assigns a value to a variable. State diagram: A graphical representation of a set of variables and the values to which they refer.
- **Keyword:** A reserved word that is used by the compiler to parse a program; you cannot use keywords like `if`, `def`, and `while` as variable names.
- **Operator:** A special symbol that represents a simple computation like addition, multiplication, or string concatenation.
- **Operand:** One of the values on which an operator operates. Expression: A combination of variables, operators, and values that represents a single result value.
- **Evaluate:** To simplify an expression by performing the operations in order to yield a single value.
- **Integer division:** An operation that divides one integer by another and yields an integer. Integer division yields only the whole number of times that the numerator is divisible by the denominator and discards any remainder.
- **Rules of precedence:** The set of rules governing the order in which expressions involving multiple operators and operands are evaluated.
- **Concatenate:** To join two operands end-to-end.
- **Composition:** The ability to combine simple expressions and statements into compound statements and expressions in order to represent complex computations concisely.

- **Comment:** Information in a program that is meant for other programmers (or anyone reading the source code) and has no effect on the execution of the program.
 - **Modulus operator:** An operator, denoted with a percent sign (%), that works on integers and yields the remainder when one number is divided by another.
 - **Boolean expression:** An expression that is either true or false.
 - **Comparison operator:** One of the operators that compares two values: ==, =, >, <, >=, and <=.
 - **Logical operator:** One of the operators that combines Boolean expressions: and, or, and not.
 - **Conditional statement:** A statement that controls the flow of execution depending on some condition.
 - **Condition:** The Boolean expression in a conditional statement that determines which branch is executed.
 - **Compound statement:** A statement that consists of a header and a body. The header ends with a colon (:). The body is indented relative to the header.
 - **Block:** A group of consecutive statements with the same indentation.
 - **Body:** The block in a compound statement that follows the header.
 - **Nesting:** One program structure within another, such as a conditional statement inside a branch of another conditional statement.
 - **Recursion:** The process of calling the function that is currently executing.
 - **Base case:** A branch of the conditional statement in a recursive function that does not result in a recursive call.
 - **Infinite recursion:** A function that calls itself recursively without ever reaching the base case. Eventually, an infinite recursion causes a runtime error.
 - **Prompt:** A visual cue that tells the user to input data.
 - **Multiple assignment:** Making more than one assignment to the same variable during the execution of a program.
 - **Iteration:** Repeated execution of a set of statements using either a recursive function call or a loop.
-

- **Loop:** A statement or group of statements that execute repeatedly until a terminating condition is satisfied.
 - **Infinite loop:** A loop in which the terminating condition is never satisfied.
 - **Body:** The statements inside a loop.
 - **Loop variable:** A variable used as part of the terminating condition of a loop.
 - **Tab:** A special character that causes the cursor to move to the next tab stop on the current line.
 - **Newline:** A special character that causes the cursor to move to the beginning of the next line.
 - **Cursor:** An invisible marker that keeps track of where the next character will be printed.
 - **Escape sequence:** An escape character (\) followed by one or more printable characters used to designate a nonprintable character.
 - **Encapsulate:** To divide a large complex program into components (like functions) and isolate the components from each other (by using local variables, for example).
-

- **Generalize:** To replace something unnecessarily specific (like a constant value) with something appropriately general (like a variable or parameter). Generalization makes code more versatile, more likely to be reused, and sometimes even easier to write.
 - **Development plan:** A process for developing a program. In this chapter, we demonstrated a style of development based on developing code to do simple, specific things and then encapsulating and generalizing.
 - **Compound data type:** A data type in which the values are made up of components, or elements, that are themselves values.
 - **Traverse:** To iterate through the elements of a set, performing a similar operation on each.
 - **Index:** A variable or value used to select a member of an ordered set, such as a character from a string.
 - **Slice:** A part of a string specified by a range of indices.
-
- **Mutable:** A compound data types whose elements can be assigned new values.
 - **Counter:** A variable used to count something, usually initialized to zero and then incremented.
 - **Increment:** To increase the value of a variable by one.
 - **Decrement:** To decrease the value of a variable by one.
 - **Whitespace:** Any of the characters that move the cursor without printing visible characters. The constant string. Whitespace contains all the whitespace characters.

THANK YOU



```
def sum(list_name):  
    value = 0  
    for values in list_name:  
        value += values  
    return value  
  
p = [1,2,3,4]  
p_sum = sum(p)  
print(p_sum)  
  
def multiply(list_name):  
    value = 1  
    for values in list_name:  
        value *= values  
    return value  
  
p = [1,2,3,4]  
p_multiply = multiply(p)  
print(p_multiply)
```

```

10
24

def word_count(str):
    counts = dict()
    words = str.split()

    for word in words:
        if word in counts:
            counts[word] += 1
        else:
            counts[word] = 1

    return counts

print( word_count('the quick brown fox jumps over the lazy dog.'))

{'the': 2, 'quick': 1, 'brown': 1, 'fox': 1, 'jumps': 1, 'over': 1,
'lazy': 1, 'dog.': 1}

print("Enter the details of the student")
name,usn,semester=input("Enter Student Name, USN, Semester:
").split(',')
usn=int(usn)
semester=int(semester)
li=[]
li.append(name)
li.append(usn)
li.append(semester)
print("Elements of a list:",li)

Enter the details of the student
Enter Student Name, USN, Semester: x,1,2
Elements of a list: ['x', 1, 2]

def main():
#Generating a size-million list, whose prefixes we will search
    numList = []
    for i in range(0, 1000000):
        numList.append(random.random())

    #Sorting this list
    numList.sort()

    #The input size n can range from hundred thousand to a million,
    #in increments of fifty thousand
    for n in range(100000, 1050000, 50000):
        sumBinaryTime = 0 # keeps track of the total time, over 100
        trials, that binarySearch takes
        sumLinearTime = 0 # keeps track of the total time, over 100

```

trials, that linearSearch takes

```
#We do 100 trials for each value of n
for i in range(0, 100):
    #Generate a number to search for
    x = random.random()

    #Compute the time it takes for binarySearch
    sbt = time.time()
    binaryFound = recursiveBinarySearch(numList, 0, n-1, x)
    ebt = time.time()
    elapsedBinaryTime = ebt - sbt

    #Compute the time it takes for linearSearch
    slt = time.time()
    linearFound = linearSearch(numList, x, n)
    elt = time.time()
    elapsedLinearTime = elt - slt

    sumBinaryTime += elapsedBinaryTime
    sumLinearTime += elapsedLinearTime
#end for-i: End of 100 trials

print(str(sumBinaryTime) + " " + str(sumLinearTime))
```